MetricMind — Agentic Semantic BI Engine

Full Project Documentation • Architecture • Problem Solving • Coding Logic • Real-World Scenarios

Purpose: A professional, implementation-oriented explanation of how MetricMind solves the enterprise trust problem in Generative-AI-powered Business Intelligence. The document emphasizes why each component exists, what happens after every step, how the prototype works, what a production system would do, and how to explain the project in interviews.

ATS note: This document is ATS-optimized through clear project terminology and relevant technical keywords. No ATS system can honestly guarantee a literal 100% score because matching depends on the specific job description and recruiter system.

1. Executive Summary

MetricMind is an Agentic Semantic BI Engine designed to answer executive business questions such as: “Why did our European margins drop last quarter?” The central design decision is to prevent the LLM from inventing business metrics or freely writing warehouse SQL. Instead, the LLM acts as an orchestrator while a governed Semantic Layer owns the authoritative definitions of Revenue, Cost, Margin, Churn and other business metrics.

The system combines Generative AI / LLM orchestration, semantic modeling, Snowflake or Databricks, dbt, Cube.dev or dbt Semantic Layer, LangChain, Python, Pandas/NumPy and a Next.js conversational BI interface. The prototype demonstrates semantic query validation, deterministic calculations, query-cost governance, multi-step root-cause analysis, visualization and audit logging.

Core principle: the LLM decides what analytical question to ask; governed semantic infrastructure decides what the metric means and how it is calculated.

2. The Main Business Problem

2.1 What companies actually struggle with

Traditional BI is reliable but often requires analysts to build dashboards or write SQL. Generative AI makes analytics conversational, but a raw text-to-SQL system creates a new risk: a fluent answer can still be analytically wrong.

- Incorrect joins can duplicate revenue and inflate totals.
- The same business metric may be calculated differently by different analysts.
- Quarter definitions, fiscal calendars and filters may be misunderstood.
- An LLM may use a technically valid column that does not represent the approved business definition.
- An unrestricted agent may generate an expensive warehouse query.
- Users may receive an answer without knowing which definition, filters or query produced it.
- Executives need explanations and root causes, not merely a number.

2.2 The dangerous failure mode

Imagine the warehouse contains revenue, discounts, returns, shipping cost and several product tables. An executive asks for margin. A raw LLM might join tables, calculate a margin from the wrong cost fields, or apply a filter incorrectly. The generated SQL can run successfully and still be wrong. This is more dangerous than a syntax error because the result looks credible.

2.3 MetricMind's answer

MetricMind separates language understanding from metric authority. The agent converts natural language into an approved semantic request. The semantic layer provides metric definitions and allowed dimensions. A governance gateway validates the request, estimates cost and blocks unsafe queries. The warehouse executes only governed analytical work.

3. Why We Built This Architecture

Problem	Design Choice	Why It Matters
LLM can hallucinate SQL	Semantic API instead of unrestricted SQL	Reduces incorrect joins and uncontrolled logic
Metric inconsistency	Central semantic definitions	Revenue/Margin are defined once
Expensive exploration	Query-cost governance	Controls warehouse spend and latency
No root-cause capability	Multi-step agent workflow	Moves from KPI change to contributing factors
Low trust	View metric definition/API/audit	Makes reasoning and provenance inspectable
Poor executive experience	Conversational BI + charts	Turns analysis into an understandable decision-support workflow

4. Real-World Scenario

Scenario: European Margin Drop

Executive question: “Why did our European margins drop in Q3?”

A useful system should not stop at “margin fell from 24% to 18%.” It should establish the metric definition, compare the relevant periods, identify the major cost drivers, and explain the result.

- Step 1 — Interpret intent: geography = Europe, period = Q3, KPI = margin.
- Step 2 — Retrieve governed metric definition: Margin = SUM(Gross Profit) / SUM(Revenue).
- Step 3 — Run a governed semantic query.
- Step 4 — Detect that Q3 margin is below the comparison baseline.
- Step 5 — Ask secondary governed queries for material cost and shipping cost.
- Step 6 — Compare contribution of the cost drivers.

- Step 7 — Generate an executive explanation and chart.
- Step 8 — Record the semantic version, request, cost estimate and result hash in an audit log.

Example outcome: The analysis could conclude that European Q3 margin declined primarily because material cost increased and shipping cost also rose. The exact percentages should come from the connected production data; the prototype uses controlled mock corporate data to demonstrate the workflow.

5. Architecture — What Each Layer Does

Business User → Natural Language → Agentic Orchestrator → Governance Gateway → Semantic Layer → Snowflake/Databricks → Results → Root-Cause Analysis → Conversational BI

5.1 Business User

The user asks a business question in natural language. The user should not need to know table names, joins or SQL syntax.

5.2 Agentic Orchestrator

LangChain and an LLM such as Llama 3 can be used to interpret the question and select approved tools. The LLM is responsible for intent understanding and orchestration, not for inventing authoritative metric formulas.

5.3 Governance Gateway

This layer checks allowed metrics, dimensions, filters, result size and estimated query cost before warehouse execution.

5.4 Semantic Layer

Cube.dev or dbt Semantic Layer stores business definitions, dimensions, relationships and approved analytical semantics. This is the source of truth for metric meaning.

5.5 Warehouse

Snowflake or Databricks stores and processes enterprise data. In the prototype, Pandas acts as a lightweight mock analytical engine so the complete concept can run without enterprise credentials.

5.6 Conversational BI

Next.js with Tremor/ECharts can present the answer, supporting data and dynamic charts. Transparency controls can expose the semantic API request and approved SQL generated by the semantic layer.

6. Why These Technologies Were Used

Technology	Why Used	Role in MetricMind
Python	Fast for data science prototypes and orchestration	Prototype engine, validation, testing and analysis
Pandas	Tabular analytics and transformations	Mock warehouse analytics and metric calculations
NumPy	Numerical operations	Supporting numerical calculations
SQL	Enterprise analytical language	Production warehouse execution

		through governed semantic tooling
Snowflake / Databricks	Scalable analytical storage/compute	Production warehouse/lakehouse
dbt	Transformation and semantic modeling workflow	Raw-to-curated data and governed models
Cube.dev / dbt Semantic Layer	Central business metric definitions	Metric authority and semantic API
LangChain	Tool-based LLM orchestration	Agent workflow and tool calling
Llama 3 / enterprise LLM	Natural-language understanding	Intent interpretation and orchestration
Next.js	Modern web application framework	Conversational BI frontend
Tremor / ECharts	Analytics visualization	Charts and executive-friendly output
JSON	Structured interface	Semantic query contract and API payloads

7. Data Model Used in the Prototype

The prototype uses a deliberately small corporate dataset so the logic is understandable and reproducible.

```
dimensions: geography, quarter, product
measures: revenue, material_cost, shipping_cost, other_cost, units

derived:
total_cost = material_cost + shipping_cost + other_cost
gross_profit = revenue - total_cost
margin = gross_profit / revenue
```

Example dimensions: Europe, North America, Asia Pacific; Q1–Q4; Software, Hardware and Services.

8. Semantic Metric Definitions — The Most Important Part

The semantic layer prevents every user, analyst or LLM from creating its own interpretation of a KPI.

```
Revenue      = SUM(revenue)
Total Cost   = SUM(material_cost + shipping_cost + other_cost)
Material Cost= SUM(material_cost)
Shipping Cost= SUM(shipping_cost)
Margin       = SUM(gross_profit) / SUM(revenue)
Units        = SUM(units)
```

Why Margin is written as SUM(Gross Profit) / SUM(Revenue): calculating row-level margins and then averaging them can produce a different answer when transaction sizes differ. The approved aggregate formula makes the business definition explicit.

9. Natural Language → Semantic Query

A user asks: “Why did our European margins drop last quarter?” The agent should extract intent, but it should not manufacture table joins.

```
{
  "metrics": ["margin"],
  "dimensions": ["geography"],
  "filters": {"quarter": "Q3"}
}
```

The semantic layer validates that margin, geography and Q3 are supported. It then compiles the governed request into the appropriate analytical execution plan.

10. Coding Logic — Prototype Core

10.1 Semantic query validation

```
class SemanticQueryError(Exception):
    pass

def validate_query(metrics, dimensions, filters):
    for metric in metrics:
        if metric not in SEMANTIC_LAYER["measures"]:
            raise SemanticQueryError("Unknown metric")
    for dim in dimensions:
        if dim not in SEMANTIC_LAYER["dimensions"]:
            raise SemanticQueryError("Unknown dimension")
    for dim, value in filters.items():
        if dim not in SEMANTIC_LAYER["dimensions"]:
            raise SemanticQueryError("Unknown filter")
    return True
```

Main idea: fail closed. If the request is outside the approved semantic contract, the system rejects it instead of guessing.

10.2 Governed execution

```
MAX_DIMENSIONS = 2
MAX_RESULT_ROWS = 100
MAX_COST_UNITS = 20

def governed_execute(query):
    validate_query(...)
    estimated_cost = estimate_cost(query)
    if estimated_cost > MAX_COST_UNITS:
        raise SemanticQueryError("Query exceeds governance budget")
    result = execute_semantic_query(query)
    if len(result) > MAX_RESULT_ROWS:
        raise SemanticQueryError("Result too large")
    return result
```

This is important for enterprise AI because agentic systems can otherwise repeatedly explore data or generate expensive analytical workloads.

10.3 Deterministic metric test

```
results = []
for _ in range(10):
```

```
        results.append(execute_semantic_query(
            metrics=["margin"],
            dimensions=[],
            filters={"geography": "Europe", "quarter": "Q3"}
        ))
    assert all(x == results[0] for x in results)
```

The purpose is not merely unit testing code. It demonstrates that a governed metric request produces reproducible results instead of stochastic LLM-generated definitions.

11. Multi-Step Agentic Root-Cause Analysis

Agentic behavior becomes valuable when one question requires multiple controlled analytical steps.

```
Question
   ↓
Measure margin
   ↓
Compare period/baseline
   ↓
If margin declines:
   ├── Query material cost
   ├── Query shipping cost
   └── Query other cost
   ↓
Rank contributing drivers
   ↓
Generate executive explanation
```

The agent does not get unlimited freedom. Every step still goes through approved semantic tools and governance controls.

12. What Happens After the Output?

The first KPI result is an intermediate analytical artifact, not necessarily the final answer.

- Validate the metric and comparison.
- Identify whether a meaningful change occurred.
- Select approved breakdowns relevant to the metric.
- Execute secondary semantic queries.
- Calculate contribution or directional impact.
- Generate a concise explanation.
- Return supporting data and visualization.
- Log the request for auditability.

This transforms the system from a text-to-SQL chatbot into an analytical decision-support agent.

13. Output Example

Executive answer structure

```
Answer:
European Q3 margin declined versus the selected baseline.
```

```
Primary KPI:
Margin = governed Gross Profit / Revenue definition.

Drivers:
1. Material cost — major adverse contributor.
2. Shipping cost — secondary adverse contributor.
3. Other cost — remaining contribution.

Evidence:
Governed semantic queries + supporting breakdown.

Governance:
Approved metric, allowed dimensions, bounded query cost.

Audit:
Request ID, semantic version, timestamp and result hash.
```

The prototype can render the supporting analysis as tables/charts. A production system would connect these results to the enterprise warehouse and semantic platform.

14. Visualization

A conversational BI response should make the change visible. Recommended visuals include margin by quarter, cost-driver contribution, geography comparison and product-level breakdown.

- Line chart: margin over time.
- Bar chart: material vs shipping vs other cost.
- Grouped bar: Europe versus other regions.
- Waterfall-style explanation: revenue → costs → gross profit → margin.

15. Audit Logging and Trust

Every analytical request should be traceable. The prototype records request ID, UTC timestamp, natural-language question, semantic query, semantic-layer version, estimated cost, duration and result hash.

```
audit_event = {
    "request_id": request_id,
    "question": question,
    "semantic_query": semantic_query,
    "semantic_layer_version": "v1",
    "estimated_cost": estimated_cost,
    "result_hash": result_hash
}
```

This supports reproducibility, debugging, governance reviews and regulated enterprise environments.

16. Security and Governance

- SSO and role-based access control (RBAC).
- Row-level security (RLS) for user-specific data access.
- Read-only analytical warehouse credentials for the agent.
- Secrets stored in a secure secrets manager.

- Query timeouts and resource budgets.
- Maximum result size and controlled dimensions.
- Audit logging and semantic-version tracking.
- Prompt/tool permissions that restrict the agent to approved capabilities.

The most important security principle is least privilege: the AI should have only the data and tools required for its task.

17. Production Cube.dev Example

```
cube: Sales
sql_table: analytics.fct_sales

measures:
  revenue:
    type: sum
    sql: revenue

  material_cost:
    type: sum
    sql: material_cost

  shipping_cost:
    type: sum
    sql: shipping_cost

  margin:
    type: number
    sql: SUM(gross_profit) / NULLIF(SUM(revenue), 0)

dimensions:
  geography:
    sql: geography
    type: string

  quarter:
    sql: quarter
    type: string
```

This illustrates the separation of metric authority from the LLM. The LLM can request margin, but the semantic system controls the definition.

18. Production Agent Tools

```
get_semantic_schema()
get_metric_definition(metric)
query_semantic_layer(metrics, dimensions, filters)
```

These tools are safer than exposing a generic execute_sql tool. Tool contracts can enforce allowed fields, filters, query budgets and permissions.

19. Example Real-World Scenarios

Industry	Question	Agent Workflow	Business Value

Retail	Why did gross margin fall in South India?	Margin → region → product → discount/cost drivers	Merchandising and supply-chain action
SaaS	Why did churn increase this quarter?	Churn → segment → plan → cohort/region	Customer retention strategy
Manufacturing	Why did production cost increase?	Cost → plant → product → material/energy/labor	Operations optimization
Banking	Why did delinquency rise?	Delinquency → portfolio → product → customer segment	Risk investigation
E-commerce	Why did contribution margin decline?	Margin → marketplace/channel → shipping/returns/discounts	Pricing and fulfillment decisions
Healthcare analytics	Why did operational cost increase?	Cost → facility/service line → staffing/supply drivers	Resource planning

20. What We Actually Implemented in the Prototype

- Created mock corporate analytical data.
- Implemented derived financial metrics.
- Defined a semantic-layer registry.
- Validated metrics, dimensions and filters.
- Executed semantic queries without requiring raw user SQL.
- Added query-cost and result-size governance.
- Tested deterministic metric behavior repeatedly.
- Built natural-language intent parsing as a controlled prototype.
- Implemented multi-step cost-driver/root-cause analysis.
- Generated executive explanations.
- Created dynamic visualizations.
- Implemented audit logging and result hashing.
- Added negative/security rejection tests.
- Included production-oriented Cube YAML, agent tool contracts and frontend response design.

21. What Is Prototype vs Production?

Area	Prototype	Production Direction
Data	Mock Pandas dataset	Snowflake/Databricks
Intent parsing	Deterministic parser	Enterprise LLM + LangChain
Semantic layer	Python registry + example YAML	Cube.dev/dbt Semantic Layer
UI	Notebook/demo visualizations	Next.js + Tremor/ECharts

Security	Validation/rejection logic	SSO/RBAC/RLS/secrets
Observability	Audit records	Centralized telemetry and monitoring

22. Testing Strategy

- Unit tests for metric calculations.
- Semantic contract tests.
- Invalid metric/dimension/filter rejection tests.
- Deterministic repeated-execution tests.
- Query cost limit tests.
- Maximum result-size tests.
- Data-quality tests for missing/invalid values.
- Semantic regression tests when metric definitions change.
- Integration tests against the semantic API.
- End-to-end tests from natural-language question to chart.

23. Data Quality Controls

Governed AI cannot compensate for unreliable source data. Production pipelines should validate freshness, duplicates, nulls, accepted ranges, referential integrity and reconciliation against trusted finance/operations totals.

- Freshness checks before analytics.
- Schema drift detection.
- Duplicate detection.
- Null and invalid-value checks.
- Source-to-target reconciliation.
- Metric-level regression monitoring.

24. Performance and Cost

Agentic analytics can create multiple tool calls. Cost governance therefore belongs in the architecture, not as an afterthought.

- Limit dimensions per request.
- Limit result rows.
- Estimate query complexity.
- Apply warehouse resource budgets.
- Use caching where safe.
- Prefer aggregated semantic queries over raw-row exploration.
- Set agent step/time limits.
- Monitor query latency and warehouse consumption.

25. Interview Explanation — 60 Seconds

“MetricMind is an Agentic Semantic BI Engine that solves the trust problem in LLM-powered analytics. Instead of allowing an LLM to freely generate SQL, I designed the LLM as an orchestrator that calls a governed semantic layer. The semantic layer owns definitions such as Revenue and Margin, while a governance gateway validates metrics, dimensions, filters, result size and query cost. I built a Python prototype using Pandas and NumPy with mock corporate data, deterministic metric tests, multi-step root-cause analysis, visualizations and audit logging. The production architecture integrates Snowflake or Databricks, dbt or Cube.dev, LangChain, an enterprise LLM and a Next.js BI interface. The key idea is to combine Generative AI flexibility with enterprise data governance and reproducibility.”

26. Common Interview Questions and Strong Answers

Q: Why not let the LLM generate SQL directly?

A: Because syntactically valid SQL can still be semantically wrong. The semantic layer centralizes business definitions and reduces hallucinated joins and metric logic.

Q: Why use a semantic layer?

A: It provides a governed source of truth for metrics, dimensions, filters and business logic.

Q: What is the role of the LLM?

A: Intent understanding, planning and tool orchestration. It does not own authoritative metric definitions.

Q: How do you control cost?

A: A governance gateway estimates complexity and enforces dimension, row, time and resource limits.

Q: How do you make the result trustworthy?

A: Deterministic metric definitions, validation, semantic versioning, audit metadata, repeatability and transparent evidence.

Q: What makes it agentic?

A: The system can perform multiple controlled analytical steps, such as KPI analysis followed by cost-driver breakdowns.

Q: Why Snowflake/Databricks?

A: They provide scalable analytical storage and compute for enterprise workloads.

Q: Why LangChain?

A: It provides practical abstractions for tool calling, orchestration and agent workflows.

Q: Why Python/Pandas?

A: They allow rapid prototyping, data transformation and reproducible analytical logic.

Q: What is the biggest production challenge?

A: Enterprise access control, semantic governance, data quality, cost control, observability and reliable integration with the warehouse/semantic platform.

27. Resume-Ready Project Description

```
MetricMind – Agentic Semantic BI Engine
Designed and implemented a governed conversational BI architecture combining Generative AI,
```

```
semantic metrics, query governance, root-cause analysis, visualization and auditability.
Built a Python/Pandas/NumPy prototype with semantic metric validation, deterministic margin
calculations, query-cost controls, multi-step cost-driver analysis, dynamic visualizations and
audit logging; designed production integration with Snowflake/Databricks, dbt/Cube.dev,
LangChain, Llama 3 and Next.js.
```

ATS keywords: Python, SQL, Pandas, NumPy, Data Science, Generative AI, LLM, Agentic AI, LangChain, Semantic Layer, Business Intelligence, Data Modeling, Snowflake, Databricks, dbt, Cube.dev, Next.js, ECharts, Data Governance, Query Optimization, Data Quality, Testing, Audit Logging, Enterprise Security, Root-Cause Analysis.

28. Portfolio / GitHub Structure

```
MetricMind/
├── notebooks/
│   └── MetricMind_Agentic_Semantic_BI_Engine.ipynb
├── data/
│   └── mock_corporate_sales.csv
├── semantic/
│   ├── metric_registry.py
│   └── sales_cube.yml
├── agent/
│   ├── orchestrator.py
│   ├── tools.py
│   └── prompts/
├── governance/
│   ├── validator.py
│   └── cost_controls.py
├── analytics/
│   └── root_cause.py
├── ui/
│   └── nextjs-app/
├── tests/
│   ├── test_metrics.py
│   ├── test_governance.py
│   └── test_semantic_contract.py
└── README.md
```

29. Final Technical Takeaways

- The core problem is not SQL generation; it is trustworthy business meaning.
- The semantic layer is the source of metric truth.
- The LLM is an orchestrator, not the system of record for business logic.
- Governance must exist before warehouse execution.
- Agentic workflows should be bounded and tool-driven.
- Root-cause analysis requires controlled follow-up queries.
- Auditability is essential for enterprise adoption.
- Data quality is foundational.
- The prototype proves the architecture; production requires enterprise infrastructure and controls.

30. Final Project Statement

MetricMind demonstrates how Generative AI can be integrated into enterprise analytics without surrendering control of business definitions. The project deliberately separates language intelligence from analytical authority: the LLM

interprets the question, the semantic layer defines what the metric means, governance controls what the agent is allowed to query, the warehouse performs scalable computation, and the BI interface explains the result. This separation creates a practical path toward trustworthy, auditable and cost-aware Agentic BI.

31. High-Level Learning Outcomes

- Enterprise data architecture and semantic modeling
- Generative AI and LLM tool orchestration
- Agentic workflow design
- Data governance and query safety
- Business metric design
- Root-cause analytical reasoning
- Python/Pandas/NumPy implementation
- SQL and analytical warehouse concepts
- Visualization and conversational BI
- Testing, observability and auditability
- Production architecture thinking

32. Important Disclaimer for Portfolio/Resume Use

Describe prototype capabilities as implemented and production integrations as designed/reference architecture unless the corresponding enterprise components have actually been deployed. This distinction makes the project credible in technical interviews and avoids overstating production experience.